

1. Install Anaconda & create virtual environment with TensorFlow

1. Download and install Anaconda (or Miniconda).
 2. Open terminal / Anaconda Prompt.
 3. Create a virtual env: `conda create -n dl-lab python=3.10 -y`.
 4. Activate it: `conda activate dl-lab`.
 5. Install TensorFlow: `pip install tensorflow` (or `conda install -c conda-forge tensorflow`).
 6. Install helper packages: `pip install numpy pandas matplotlib scikit-learn jupyter`.
 7. Launch Jupyter / test: run a small script that imports tensorflow and prints its version.
-

2. Build a shallow neural network for Fashion MNIST

1. Load Fashion MNIST dataset (from `tensorflow.keras.datasets`).
 2. Normalize pixel values to range `[0,1]`.
 3. Flatten `28×28` images to 1D vectors.
 4. Define model: `Input → Dense(hidden, e.g. 128, ReLU) → Dense(output=10, Softmax)`.
 5. Compile model with `loss=sparse_categorical_crossentropy`, `optimizer=adam`, `metrics=accuracy`.
 6. Train on training set (e.g. `epochs=10`, `batch_size=32`) and validate on test set.
 7. Evaluate accuracy and inspect misclassified images.
-

3. Build a deep feedforward neural network for Fashion MNIST

1. Load and normalize the dataset.
 2. Optionally apply simple augmentations or keep flattened input.
 3. Define deeper model: multiple Dense layers (e.g. `512 → 256 → 128`) with ReLU and dropout between layers.
 4. Compile with appropriate loss & optimizer (e.g. Adam).
 5. Train with early stopping or validation monitoring.
 6. Plot training vs validation loss to check overfitting.
 7. Evaluate and compare with the shallow network.
-

4. Hyperparameter tuning (find best settings)

1. Choose hyperparameters to tune: learning rate, batch size, number of layers, units per layer, dropout.

2. Pick a search strategy: grid search (try all combos) or random search (sample combos).
 3. For each combo: build model, train on training set, evaluate on validation set.
 4. Record performance metrics for each run.
 5. Analyze results, pick the best combination (highest validation accuracy or lowest loss).
 6. Retrain final model with best hyperparameters on combined train+val (if appropriate).
 7. Report final test accuracy and note runtime/complexity.
-

5. Predict house prices (regression with neural network)

1. Load dataset (features like area, bedrooms, location; target = price).
 2. Clean data: handle missing values and encode categorical variables (one-hot or embeddings).
 3. Split into train/val/test, and scale numeric features (StandardScaler or MinMax).
 4. Define model: Input → several Dense layers with ReLU → output Dense(1) (no activation).
 5. Compile with loss=mean_squared_error (MSE) and optimizer=adam.
 6. Train and monitor validation MSE; use early stopping if overfitting.
 7. Evaluate using RMSE and MAE; visualize predicted vs actual prices.
-

6. Convolutional Neural Network (CNN) for image classification

1. Load and normalize image dataset; resize images if needed.
 2. Build CNN architecture: Conv → ReLU → Pool → Conv → Pool → Flatten → Dense → Softmax.
 3. Use batch normalization and dropout where helpful.
 4. Compile with categorical/sparse categorical crossentropy and optimizer (Adam).
 5. Train with data augmentation (random flips, rotations) to generalize better.
 6. Evaluate on test set; visualize activation maps or filters if desired.
 7. Save the trained model for inference.
-

7. Transfer learning with pre-trained CNN

1. Choose a pre-trained model (e.g., VGG16, ResNet) with weights=imagenet.
2. Load pre-trained base **without** its top classifier (include_top=False) and freeze its layers.
3. Add new top layers: GlobalAveragePooling → Dense → Softmax for your classes.

4. Compile and train only the new head first (small learning rate).
 5. Optionally unfreeze some deeper base layers and fine-tune with an even smaller learning rate.
 6. Use data augmentation and checkpoints to save best model.
 7. Evaluate and compare with training-from-scratch results.
-

8. Sentiment analysis of movie reviews (text classification)

1. Load text dataset (reviews + labels).
 2. Clean text (lowercase, remove punctuation) or use tokenizer that handles raw text.
 3. Tokenize and convert to integer sequences; pad sequences to fixed length.
 4. Define model: Embedding → (LSTM or GRU or simple RNN) → Dense → Sigmoid (binary) / Softmax (multi-class).
 5. Compile with binary/ categorical crossentropy and optimizer Adam.
 6. Train and validate; use dropout or recurrent dropout to avoid overfitting.
 7. Evaluate using accuracy, precision, recall, and confusion matrix; inspect example predictions.
-

9. Translation model using pre-trained Large Language Model (LLM)

1. Pick a pre-trained seq2seq transformer (e.g., T5, MarianMT) that supports translation.
 2. Tokenize source and target languages using the model's tokenizer.
 3. Fine-tune the model on a parallel corpus (source-target sentence pairs) using teacher forcing.
 4. Use appropriate loss (cross-entropy) and a small learning rate for fine-tuning.
 5. Validate with BLEU or METEOR scores on held-out translations.
 6. During inference, use beam search or greedy decoding to generate target sentences.
 7. Postprocess (detokenize, fix punctuation) and inspect sample translations.
-

10. Chatbot using pre-trained transformer (dialogue system)

1. Choose a conversational transformer (e.g., DialoGPT / GPT-family).
2. Prepare dialogue data (pairs of user input → bot response) for fine-tuning if needed.
3. Tokenize conversation context; consider keeping a sliding window of recent turns.
4. Fine-tune with causal language modeling loss or use the model as-is with prompt engineering.
5. Implement context handling, temperature/top-k sampling for varied responses.

6. Add safety/filters and fallback responses for unknown or unsafe queries.
7. Deploy inference endpoint and test with multi-turn conversations; collect logs for improvement.